

ADVANCED DATA MANAGEMENT 2023/2024
September 11, 2024
CLOSED BOOK

Q3. (up to 10 lines) [**only for COMPUTER ENGINEERING STUDENTS**]
Discuss and compare multi-leader and leader-less replication protocols.

Q2. (up to 10 lines)
Discuss how join operations can be executed over a NoSQL data store.

Q3. (up to 10 lines)

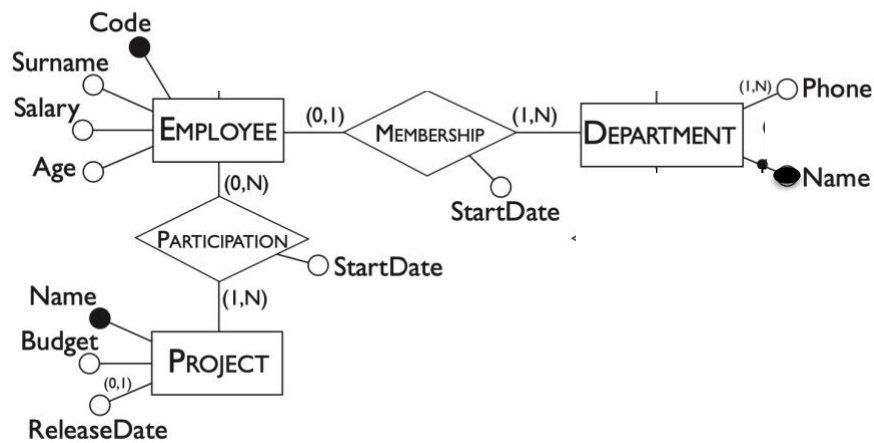
Compare common partitioning and replication approaches in aggregate-oriented and graph-based NoSQL systems.

Q4. (up to 10 lines) [**only for COMPUTER SCIENCE STUDENTS**]

Describe the main differences between RDF, RDFS, and OWL.

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following entity relationship diagram:



The identifier of a department is composed of the name and the branch it is related to.

The workload is the following:

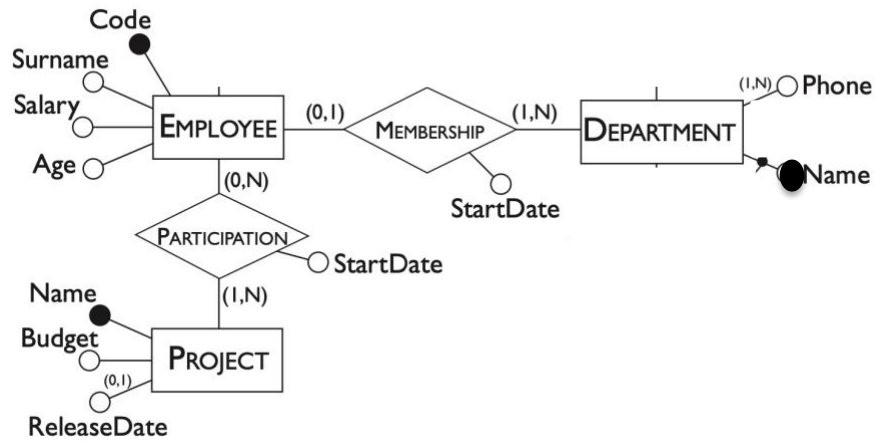
- **Query 1:** Determine the code and the salary of the employees under 24.
- **Query 2:** Determine the code and the surname of the employees that work in the department named "Research".
- **Query 3:** Determine the projects in which the employee with code "35677" works.

Based on the previous ER schema and workload:

1. Design, using the aggregate-oriented logical design methodology, the aggregates needed for answering the given workload queries. Present the output of each step of the methodology.
2. Map the obtained aggregate schemas into a set of MongoDB schemas, represented in Json-like meta-notation; for each MongoDB schema, specify the shard key and the indexes that should be created.
3. Specify Query 3 in the MongoDB language.

Exercise 2 [NoSQL graph-based data stores]

Consider the entity relationship diagram presented in Exercise 1:



1. Present the most suitable logical schema in a graph-based data store, like Neo4j (in the form of a graph pattern or of a sample instance), for the domain described by the previous diagram.
2. Formulate in Neo4j the query to determine the average salary of the employees that started to work in a department from January 2024, together with the name and the release date of the projects in which they participate.

Exercise 3 [Semantic data stores] [only for COMPUTER SCIENCE STUDENTS]

Consider the following extract from an RDF file:

```
ex:book1 ex:hasTitle "Lord of the Rings" .
ex:book1 ex:writtenBy ex:JRR Tolkien .
ex:book1 ex:publishedInYear "1954" .
ex:book1 ex:originalLanguage ex:English .
ex:book1 ex:hasGenre ex:Fantasy .
.
ex:book2 ex:hasTitle "O Alquimista" .
ex:book2 ex:writtenBy ex:Paulo Coelho .
ex:book2 ex:publishedInYear "1988" .
ex:book2 ex:originalLanguage ex:Portuguese .

ex:JRR Tolkien ex:hasNationality ex:British .
ex:Paulo Coelho ex:hasNationality ex:Brazilian .
```

- Add the class specifications for the classes **ex:Book**, **ex:Genre**, and **ex:Author**.
- Specify **ex:Author** as a subclass of **foaf:Person**.
- Specify **ex:writtenBy** as a property, and specify its range and domain.
- Identify at least one property among those in the extract above which is functional and specify it as a functional property.
- Write a SPARQL query to retrieve the titles of all books published by Tolkien between the years 1950 and 1965.
- Write a SPARQL query to retrieve the title, the author and (if available) the genre of all books published by a Portuguese author or originally in Portuguese language.

Hint (Cheat sheet RDF/S and OWL)

- instance relation `rdf:type` (classes are instances of `rdfs:Class`)
- subclass relation `rdfs:subClassOf`
- domain of a property `rdfs:domain`
- range of a property `rdfs:range`
- subproperty `rdfs:subPropertyOf`
- functional property `owl:functionalProperty`
- inverse functional property `owl:inverseFunctionalProperty`
- class disjointness `owl:disjointWith`
- instance equality and disequality `owl:sameAs` and `owl:differentFrom`